

Informe Ejercicio 1

Algoritmo Método Burbuja

El presente informe describe el desarrollo de un algoritmo que permite generar un arreglo de N elementos y ordenarlo de mayor a menor utilizando el método burbuja. El programa fue desarrollado aplicando buenas prácticas de programación basadas en el estándar PEP8, utilizando funciones, nombres descriptivos y pruebas unitarias para validar el correcto funcionamiento del algoritmo.

1. Descripción del Problema

Se requiere desarrollar un algoritmo que permita:

- Solicitar al usuario la cantidad de elementos.
- Ingresar los valores del arreglo.
- Ordenar el arreglo de mayor a menor utilizando el método burbuja.
- Mostrar el arreglo original y el arreglo ordenado.

2. Desarrollo de la Solución

El problema se resolvió mediante el uso de funciones independientes que permiten organizar mejor el código y facilitar su mantenimiento:

- ordenar_burbuja_descendente()**: realiza el ordenamiento mediante el método burbuja.
- solicitar_numeros()**: solicita los datos al usuario y los almacena en una lista.
- main()**: coordina la ejecución principal del programa.

El algoritmo compara elementos vecinos y, cuando el número de la izquierda es menor que el de la derecha, realiza un intercambio para ordenar el arreglo de mayor a menor.

3. Código Implementado

```
def ordenar_burbuja_descendente(arreglo):
    """
    Ordena un arreglo de mayor a menor usando el método burbuja.
    """
    longitud = len(arreglo)

    # Recorre todos los elementos de la lista
    for i in range(longitud):
        for j in range(0, longitud - i - 1):

            # Compara el elemento actual con el siguiente
            if arreglo[j] < arreglo[j + 1]:
                arreglo[j], arreglo[j + 1] = arreglo[j + 1], arreglo[j]

    return arreglo
```

4. Prueba de Escritorio

Se utilizó el siguiente arreglo de prueba: [5, 2, 8, 1] El algoritmo realiza comparaciones e intercambios hasta ordenar completamente el arreglo.

Pasada	Comparación	Acción	Resultado
1	5 vs 2	No se intercambia	[5, 2, 8, 1]
1	2 vs 8	Se intercambia	[5, 8, 2, 1]
1	2 vs 1	No se intercambia	[5, 8, 2, 1]
2	5 vs 8	Se intercambia	[8, 5, 2, 1]
2	5 vs 2	No se intercambia	[8, 5, 2, 1]

5. Pruebas Unitarias

Con el fin de validar el correcto funcionamiento del algoritmo, se implementaron pruebas unitarias utilizando la librería **unittest** de Python. Las pruebas permiten verificar que el algoritmo funcione correctamente en diferentes escenarios: Listas desordenadas. Listas ya ordenadas. Listas con números repetidos. Listas vacías. Listas con un solo elemento.

```
import unittest

from ejercicio1_burbuja import ordenar_burbuja_descendente

class TestOrdenamientoBurbuja(unittest.TestCase):

    def test_lista_desordenada(self):

        resultado = ordenar_burbuja_descendente ([5, 2, 8, 1])

        self.assertEqual(resultado, [8, 5, 2, 1])

    def test_lista_vacia(self):

        resultado = ordenar_burbuja_descendente([ ])

        self.assertEqual(resultado, [ ])

if __name__ == "__main__":

    unittest.main()
```

Al ejecutar las pruebas unitarias, todas las validaciones fueron exitosas, confirmando que el algoritmo realiza correctamente el ordenamiento descendente.

6. Resultado Final

Después de ejecutar el algoritmo, el arreglo queda organizado de mayor a menor: **[8, 5, 2, 1]**

7. Conclusión

El método burbuja ayudó a entender cómo ordenar una lista de números paso a paso, comparando un número con el siguiente y cambiándolos de lugar cuando era necesario. Además, las pruebas realizadas permitieron comprobar que el programa funciona correctamente en diferentes casos.